



PhishGuard-AI

Technical work report

Phishing detection & cyber-awareness platform for Cameroon

Item	Detail
Project	Bachelor final-year project — Lareine Tracy
Stack	Flask 3 + PostgreSQL · Vue 3 + TypeScript + Tailwind v4 · Claude API · scikit-learn / XGBoost
Detection model	PhishGuard-XGB v2.4.1 (hybrid ML + AI + URL intel + rules)
Report date	25 September 2026

1. Executive summary

The work started with a review of the existing code against the project documentation, followed by a gap analysis against commercial anti-phishing products (Proofpoint, Microsoft Defender for Office 365, Mimecast, Google Safe Browsing). A two-week plan was then executed: a multi-stage **automatic detection pipeline** combining machine learning, a large language model (Claude), URL reputation and sender authentication was built and wired to the public landing page, the user dashboard and the administrator dashboard. The dashboards were rebuilt on live data, including an antivirus-style scan experience, a notification centre, account settings with OTP two-step login, an admin *central command* with a human review queue and a live world threat map. Finally the visual identity was unified (logo, favicon, dark/light theme, softer authentication screens).

Area	Result
Detection	4-signal fusion pipeline with explainable verdicts and safety overrides
User side	Animated scanner, threat alerts, history, notifications, profile, avatar, OTP 2FA
Admin side	Live KPIs, traffic chart, review queue, whitelist, threat map, CSV export
UX / branding	Dark & light mode fixed, PhishGuard logo everywhere, favicon, soft auth forms
Quality	vue-tsc type-check and Vite production build pass; endpoints tested with the Flask test client

2. Architecture of the detection pipeline

Every analysis request (POST /api/v2/scan) goes through the package backend/app/pipeline:

Stage	Module	Role
1. Parser	parser.py	Normalises text/email, extracts URLs (including bare domains), phone numbers, sender
2. Sender check	sender.py	Registered domain, brand look-alike detection, whitelist, SPF/DKIM hints
3. URL intel	url_intel.py	Heuristics on each link, optional XGB URL model, geolocation for the map
4. AI analysis	ai_analyzer.py	Claude via structured output (Pydantic AIVerdict); personal data redacted before sending
5. Fusion	fusion.py	Weighted score + overrides → verdict, level, explanation, advice

Fusion weights and rules

- Weights: ML text model 0.30 · AI 0.25 · URL intelligence 0.25 · rules/heuristics 0.20.
- Override: brand spoofing or a known-bad URL forces the score to at least 85.
- Override: a whitelisted and authenticated sender caps the score at 30.
- Verdicts are normalised to *phishing* / *suspicious* / *legitimate*; the 4-level scale (Critical >80, High >60, Medium >40, Low) is kept in Level.
- If the AI is unavailable (no key, timeout) the weights are redistributed, so the service never fails.
- The XGB URL model showed a bias during tests; it is disabled by default (ENABLE_XGB_URL) and flagged in the admin ML screen until retrained.

-
- Safety: the AI may only cite official contacts (CIRT-CM 8202, alerts@cirt.cm) — invented numbers are filtered.

3. Detailed list of work done

Work item	Description	Main files
Code review & gap analysis	Compared code with the documentation; listed missing features vs. commercial products (real-time AI, URL reputation, sender authentication, human review loop, telemetry, threat map).	—
Automatic AI pipeline	New pipeline package (parser, sender, URL intel, AI, fusion) and the /api/v2 scan endpoint.	app/pipeline/*, app/api/scan.py
Live AI	Claude integration with structured output, PII redaction, timeouts and graceful fallback. Dependencies pydantic and anthropic added.	ai_analyzer.py, requirements.txt
Landing page connection	Public analyzer calls the new pipeline and shows the verdict, indicators and advice.	Analyzer.vue, analysis.service.ts
User dashboard	Antivirus-style scan animation, threat alert card, history, statistics.	DashboardView.vue, scan/ScanAnimation.vue, scan/ThreatAlert.vue
User settings	Avatar upload, name, email change, OTP two-step login.	user_profile.py, userAccount.service.ts
Notifications	Notification centre with unread badge and <i>read more</i> detail.	userNotifications.ts
Admin central command	Real KPIs, traffic chart, recent detections, system health on live data.	admin_ops.py, admin/Overview.vue, TrafficChart.vue, adminOps.ts
Human review queue	Admins confirm/overturn AI verdicts; new DB columns and migration.	ReviewQueue.vue, ReviewDrawer.vue, database/scripts/07_review_queue.sql, models.py
Whitelist	Trusted senders/domains used by the fusion override.	Whitelist.vue, admin_ops.py
Live world threat map	d3-geo world map with every detected link geolocated (ip-api.com, cached) and a results list.	ThreatMap.vue, services/geo_service.py
Dark / light mode	Fixed Tailwind v4 dark variant, theme stored in pg-theme.	main.css, composables/useTheme.ts
Model naming	Model presented as PhishGuard-XGB v2.4.1 in the UI.	admin ML screens
Branding	PhishGuard logo in scanner, header, favicon set, tab title PhishGuard-AI, lang=fr.	AppLogo.vue, index.html, public/*
Auth screens	Logo on login & register, softer design (rounded cards, pill buttons, soft inputs), registration verified by 6-digit OTP only (email link removed), password rules aligned with backend (12 chars).	auth/LoginForm.vue, RegisterForm.vue, AuthHeader.vue, auth.css, api/auth.py

Bugs fixed along the way

Problem	Fix
Python files re-indented by the editor	Restored with textwrap.dedent

Problem	Fix
Migration failed (ownership)	SQL executed as the postgres owner
AmbiguousForeignKeyError on User.analyses	Explicit foreign_keys on the relationship
Missing jsonify import in routes.py	Import added
Verdict labels mismatched between backend and UI	Normalised verdict + level
AI sentences appearing as brand names	Output filtered
Dark mode not working under Tailwind v4	@custom-variant dark added
Links missing from the map	Text recovery + bare-domain extraction
AI inventing phone numbers	Contacts restricted to CIRT-CM
Logo path broken in production build	Imported through Vite

4. Proposed next step: automatic email scanning

Today users paste a suspicious message. The next step is to let PhishGuard-AI read the mailbox itself, with the user's explicit authorisation, and scan each new email automatically.

4.1 Authorisation (OAuth 2.0, read-only)

- **Gmail:** Google Cloud project → enable Gmail API → OAuth consent screen → scope `gmail.readonly` (optionally `gmail.modify` to add a label). The existing Google login (`app/api/oauth.py`) is extended with a separate “Connect my mailbox” consent.
- **Outlook / Microsoft 365:** Azure app registration → Microsoft Graph scope `Mail.Read`.
- **Other providers** (Yahoo, Camtel...): IMAP with an app password, stored encrypted.
- Refresh tokens are encrypted (Fernet key in `.env`) in a new `mail_connections` table; the user can revoke access at any time from the Security page.

4.2 Receiving new emails

- Gmail push: `users.watch` → Google Pub/Sub → webhook POST `/api/mail/webhook` → fetch messages via `history.list`.
- Microsoft Graph: change-notification subscription on `/me/messages`.
- Fallback: a scheduled job (APScheduler/Celery) polling every 2–5 minutes.

4.3 Analysis

- Parse the raw MIME: headers (From, Reply-To, Return-Path), **Authentication-Results** (SPF, DKIM, DMARC), HTML body, links (real href vs. displayed text) and attachments (name, type, hash).
- Send to the existing pipeline — the sender stage already expects SPF/DKIM signals, so real headers make it stronger.
- Only the minimum is kept: verdict, score, sender, subject and indicators — never the full body (privacy).

4.4 Action and feedback

- Phishing: add a Gmail label *PhishGuard* ■ (or move to spam, if the user allows it) and send an in-app/SMS alert.
- Show the results in a new “My mailbox” tab of the user dashboard; high-risk emails go to the admin review queue.
- User “this is safe / this is phishing” feedback becomes training data for the next model version.

Estimated effort: about 1–2 weeks (OAuth + Gmail first, then Outlook). Note: Google requires an app verification for restricted Gmail scopes before public release; in testing mode up to 100 test users can connect.

Flow

```
Mailbox → (push / polling) → /api/mail/webhook → fetch MIME → parser + SPF/DKIM/DMARC →
PhishGuard pipeline (ML + AI + URL + rules) → verdict → label / alert / dashboard / admin
review
```

5. Known limitations, security notes and recommendations

- The XGB URL model and the text model should be retrained on a larger, balanced Cameroonian dataset.

-
- Some admin screens (integrations, parts of threat intelligence) still display demonstration data.
 - Interface translation (French/English) is planned but not yet implemented.
 - Test analyses created during development should be cleaned from the database.
 - **Security:** secrets live only in backend/.env, which must never be committed. The Gmail app password and the placeholder SECRET_KEY must be rotated; the Anthropic API key shared earlier must be revoked and replaced. The SMTP login is currently rejected by Gmail, so verification codes are not delivered until a new app password is set.
 - Add backend/instance/ (geolocation cache) to .gitignore.
 - No visual browser test campaign was performed yet; a manual walkthrough of every screen in both themes is recommended.